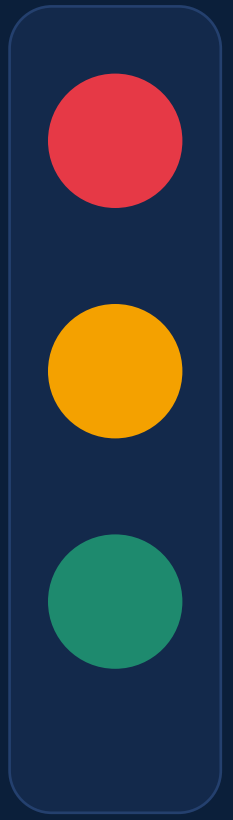
MINOR PROJECT

In partial fulfilment of internship requirements at Ediglobe

IR Remote Controlled Smart Traffic Light System with Countdown Timer

An Arduino-based system combining infrared remote control, real-time countdown display and automatic signal sequencing



Jostan Saldanha

B.Tech, Electronics & Communication Engineering (Advanced Communication Technology)

NMAMIT (Nitte Deemed to be University), Karnataka

Introduction

Traffic congestion and unsafe intersections remain a persistent challenge in growing urban and campus environments. Conventional traffic signal units are fixed-function and offer no direct, local way to override or test their behaviour, which makes teaching, prototyping and demonstration difficult.

This project builds a compact, low-cost simulation of a traffic-light controller on an Arduino Uno. An infrared (IR) remote lets an operator manually force Red, Yellow or Green, or hand control back to an automatic countdown sequence — with the remaining seconds shown live on a 7-segment display.

AT A GLANCE

Controller

Arduino Uno R3

Input

IR remote (NEC-style)

Output

3 status LEDs + 7-seg display

Modes

Manual override / Automatic

Display driver

Adafruit LED Backpack (I2C, 0x70)

Objectives

Decode IR remote signals

Capture and interpret NEC-style IR codes from a standard remote using the IRremote library.

Manual LED override

Let the user directly force Red, Green or the amber/third LED at any time via dedicated remote buttons.

Automatic countdown mode

Toggle a self-running sequence that steps the signal through Red → Amber → Green with a live countdown.

Live numeric display

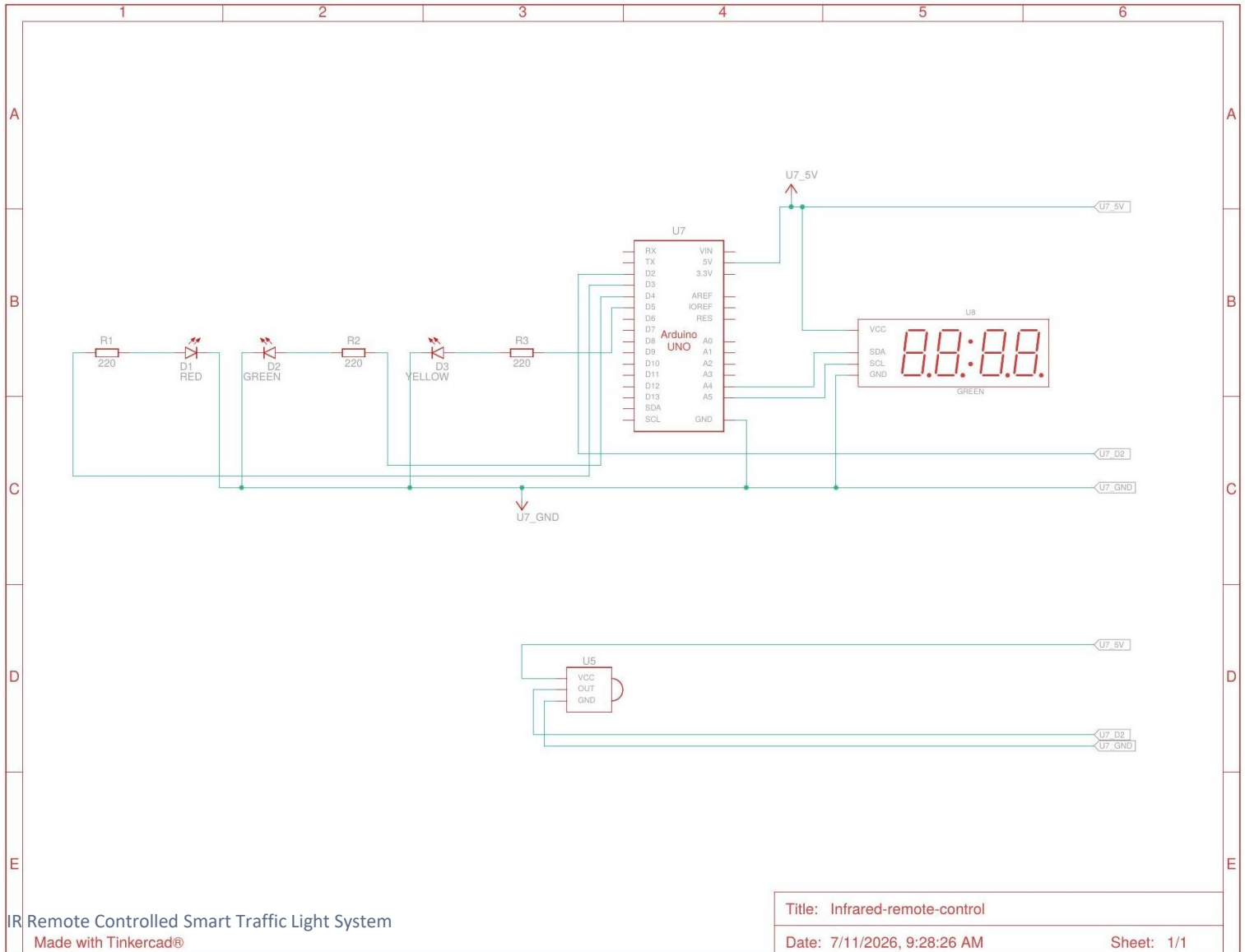
Drive a 7-segment I2C backpack to show the current countdown value or mode status in real time.

Hardware Components

Ref. Designator	Qty	Component	Role in circuit
D1	1	Red LED	Manual / “Stop” indicator
D2	1	Green LED	Manual / “Go” indicator
D3	1	Yellow LED	Manual / “Caution” indicator (driven as BLUE_LED in code)
R1, R2, R3	3	220 Ω Resistor	Current-limiting for D1–D3
U5	1	IR Sensor	Receives NEC-style remote codes
U7	1	Arduino Uno R3	Main microcontroller / logic
U8	1	7-Segment Display (0x70)	I2C countdown / status readout

Source: Component_List_IR_Remote.pdf · bom.csv

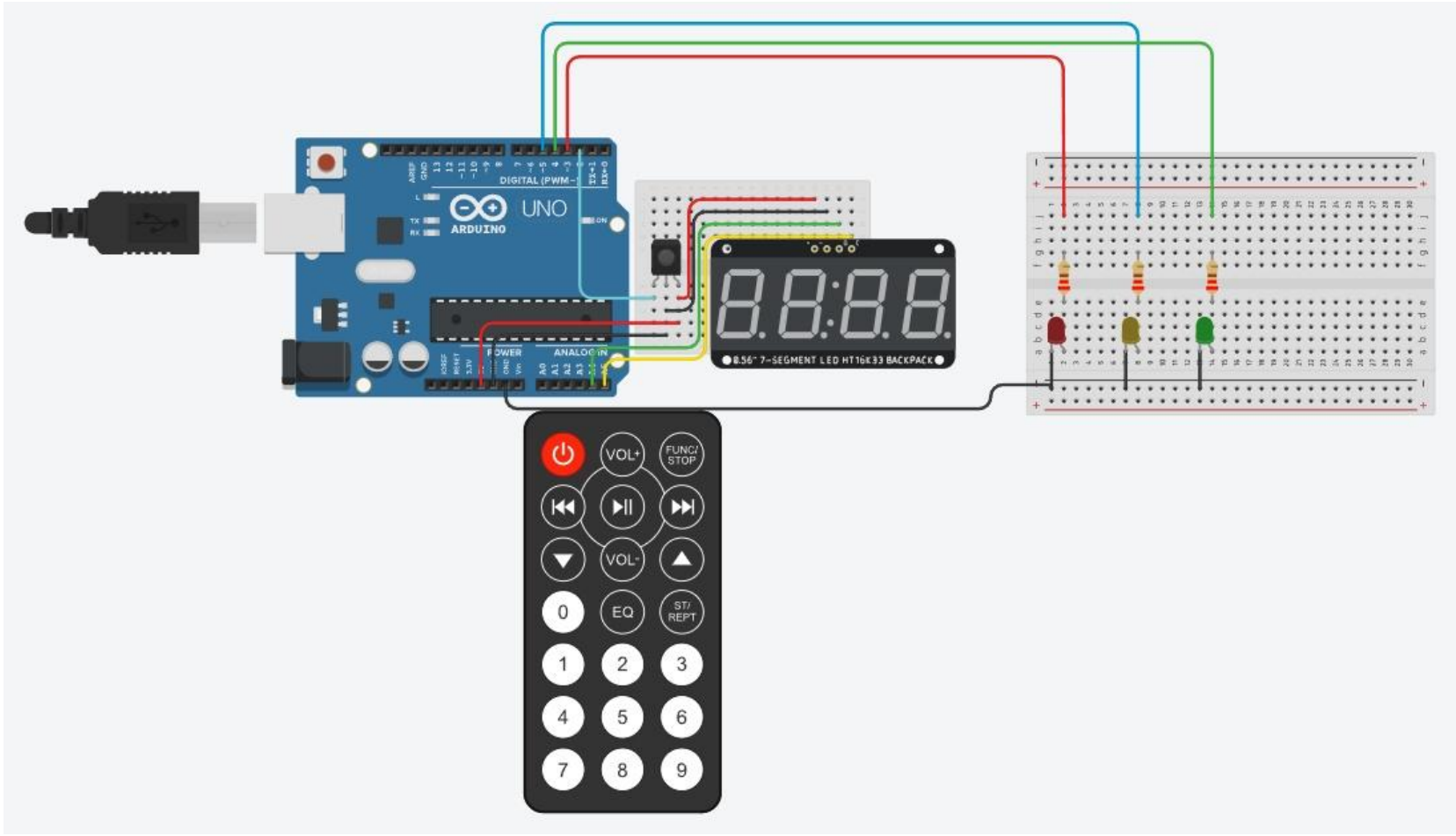
Circuit Diagram



WIRING NOTES

- IR receiver (U5) OUT → Arduino D2
- D1 (Red) via R1 → digital pin 3
- D2 (Green) via R2 → digital pin 4
- D3 (Yellow) via R3 → digital pin 5
- 7-segment display U8 → SDA / SCL (I2C, addr 0x70)
- Shared 5V and GND rails from Arduino U7

Breadboard Wiring Diagram



Tinkercad build view — Arduino Uno, IR receiver, breadboarded LEDs and 7-segment backpack

AT A GLANCE

- Red, Green, Yellow LEDs sit on the right breadboard with 220 Ω series resistors to digtal inputs of Arduino
- 7-segment backpack draws power and I2C from the Arduino's 5V/GND/SDA/SCL rails
- IR receiver module plugs into the small breadboard next to the display driver
- IR remote (bottom) is the handheld transmitter used to test manual and auto modes

IR Remote Command Mapping

Each key on the remote sends a repeating NEC-style frame. mapCodeToButton() extracts and validates the 8-bit command byte (address/inverse-address check) and returns a simple button ID that the main loop switches on.

Remote Key	Code Constant	Value	Resulting Action
Numeric 0	BTN_OFF	12	Exit auto mode, all LEDs off, display shows 0
Numeric 1	BTN_RED	16	Force Red LED, display shows 1
Numeric 2	BTN_BLUE	17	Force Yellow/third LED, display shows 2
Numeric 3	BTN_GREEN	18	Force Green LED, display shows 3
Play / Pause	BTN_PLAY	5	Toggle automatic countdown mode on/off

Operating Modes

MANUAL MODE

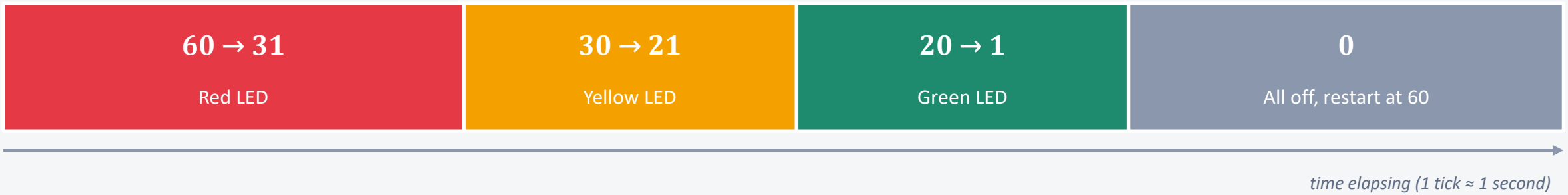
- Any of the Numeric 0–3 keys immediately sets autoMode = false.
- The matching LED function (red(), green(), or the yellow/third-LED equivalent) is called directly.
- showValue() writes 0–3 to the 7-segment display as a mode indicator.
- Gives an operator instant, deterministic override of the signal at any time.

AUTOMATIC MODE

- Play/Pause toggles autoMode; entering it resets timerCount to 60 and stamps lastTick.
- automaticMode() runs once per second (millis()-based, non-blocking).
- LED colour is chosen from timerCount: >30 Red, >20 Yellow, >0 Green, else off.
- The countdown value is mirrored on the display each tick, then decremented and auto-restarted at 60.

Countdown Timer & LED Logic

timerCount runs from 60 down to 0 in automatic mode, changing the active LED at fixed thresholds:



IMPLEMENTATION NOTE

The code names the third status LED BLUE_LED (pin 5) for readability, but the physical build wires this channel to the Yellow LED (D3) as shown in the schematic and BOM — so the amber “caution” state is what actually lights up between Red and Green. Timing uses millis() rather than delay(), so the IR receiver keeps polling for a manual-override key press even while the countdown is running.

IR Signal Decoding Flow

1

IrReceiver.decode()

Checks whether a new IR frame has arrived this loop iteration.

2

Read decodedRawData

Pulls the full 32-bit raw NEC frame from the receiver library.

3

mapCodeToButton()

Masks the lower 16 bits, checks the 0xBF00 address, then validates command / ¬inverse-command.

4

Return button ID

Yields a small integer (or –1 if invalid) consumed by the main switch statement.

5

IrReceiver.resume()

Re-arms the receiver so the next transmitted frame can be captured.

mapCodeToButton() — bom-independent NEC command extraction

```
int mapCodeToButton(unsigned long code) {
  if ((code & 0x0000FFFF) == 0x0000BF00) {
    code >>= 16;
    if (((code >> 8) ^ (code & 0x00FF)) == 0x00FF)
      return code & 0xFF;
  }
  return -1;
}
```

Key Code Highlights

automaticMode() — non-blocking, one-tick-per-second countdown driver

```
void automaticMode() {
  if (millis() - lastTick >= 1000) {
    lastTick = millis();

    if (timerCount > 30) red();
    else if (timerCount > 20) blue();   // yellow LED
    else if (timerCount > 0) green();
    else ledsOff();

    showValue(timerCount);
    timerCount--;

    if (timerCount < 0)
      timerCount = 60;   // auto-restart
  }
}
```

WHY IT'S DESIGNED THIS WAY

- **millis() over delay()** — keeps the loop non-blocking so IR key presses are never missed while counting down.
- **Threshold cascade** — a simple if/else ladder maps the countdown value directly to an LED state — easy to re-tune.
- **Single source of truth** — showValue() and the LED functions are reused identically by both manual and automatic paths.
- **Self-restarting loop** — timerCount wraps back to 60, so the signal cycle repeats indefinitely without operator input.

Results & Demonstration

The design was built and functionally verified as a Tinkercad circuit simulation, exercising both control paths end to end.

Manual override verified

Each of the four override keys (0–3) instantly switches the correct LED and updates the display, overriding any running countdown.

Automatic cycle verified

Play/Pause reliably starts a 60-second countdown that steps Red → Yellow → Green → restart, matching the coded thresholds.

Live display confirmed

The I2C 7-segment backpack (address 0x70) mirrors the countdown value and manual-mode codes without lag.

Non-blocking response

IR key presses are still accepted and acted on immediately while the automatic countdown is running.

Advantages & Applications

ADVANTAGES

- Low-cost build using only an Arduino Uno, three LEDs and an I2C display
- Manual override gives instant operator control for demos or emergencies
- Non-blocking millis() timing keeps the system responsive at all times
- Simple, well-commented code base that is easy to extend or re-tune

APPLICATIONS

- Classroom / lab demonstration of embedded control and IR communication
- Scaled-down teaching model for junction signal timing concepts
- Base platform for a manually-assisted crossing at a low-traffic junction
- Starting point for multi-junction or sensor-based smart traffic research

Future Scope

01

Vehicle density sensing

Add IR/ultrasonic or camera-based density sensing so green-phase duration adapts to real traffic instead of a fixed 60-second cycle.

02

Multi-junction coordination

Extend from a single signal to a synchronised, wirelessly-linked network of junctions.

03

Pedestrian & emergency priority

Add a pedestrian call button and an emergency-vehicle override channel on top of the existing IR interface.

04

Data logging & dashboard

Log mode changes and timings over serial/Wi-Fi (e.g. ESP32) to a simple dashboard for analysis.

Conclusion

This project demonstrates a compact, fully-working Arduino platform that fuses IR remote control, real-time LED signalling and a live countdown display into one coherent traffic-light controller. Manual override and an automatic, self-restarting timer coexist cleanly thanks to non-blocking, millis()-based scheduling — giving a reliable foundation that is easy to extend toward sensor-driven, adaptive traffic management.

Arduino Uno R3

IRremote library

I2C 7-segment display

Non-blocking control loop

Thank you